

터무니 Agent 자율성 분석

작성: 2026-06-05 대상: 발표 시 "Agent 자율성" 질문 대비 + 기술 깊이 어필

1. Agent 자율성 6단계 분류

Level	명칭	특징	예시
0	Rule-based	정해진 if-else 규칙	switch문 챗봇
1	Reactive	정의된 응답 (메모리 X)	단순 FAQ 챗봇
2	Tool-using	LLM이 도구 중 선택	function calling, gpt-4 plugins
3	Planning	LLM이 자체 계획 수립 + 단계 결정	ReAct, OpenAI Assistants
4	Self-correcting	실행 결과 평가 + 재시도/방향 전환	AutoGen, CrewAI, Devin
5	Fully Autonomous	목표만 주어진다면 끝까지 자율 + 학습	AGI 영역

2. 터무니의 현재 위치 — Level 2 + 부분적 Level 3

LLM이 자율 판단하는 8개 결정 포인트

사용자 입력

↓

[결정 1] Planner Agent (gpt-4o-mini)

"이 질문의 의도는? execution_plan은?"

↓

[결정 2] Domain Router (legal-rag-agent)

"이 모드면 어떤 RAG 도메인을 검색?"

→ real_estate=[contract,case] / business=[law,ordinance,contract]

↓

[결정 3] Query Rewriter (gpt-4o-mini)

"자연어 → 검색 키워드 변환"

→ "강남 카페 차리려고요" → "강남구 휴게음식점 인허가 규제"

↓

[Hybrid Search 실행]

↓

[결정 4] LLM Reranker (gpt-4o-mini)

"20개 후보 중 어느 5개가 가장 관련?"

↓

[결정 5] Self-RAG (gpt-4o-mini)

"이 결과로 답변 가능? confidence 점수는?"

↓

[결정 6] Corrective RAG (gpt-4o-mini)

"부족하다면 어떤 키워드를 더해서 재검색?"

↓

[결정 7] Relevance Filter (gpt-4o-mini)

"Naver 12건 중 진짜 관련된 것 골라"

↓

[결정 8] Decision Agent (gpt-4o-mini)

"모든 데이터 종합: verdict/reasons/actions/red_flags"

↓

최종 응답

→ 8개 결정 모두 LLM이 자율 수행. 규칙 기반 시스템은 사람이 모든 결정을 사전 정의.

3. 자율성의 근거 (코드 위치)

#	자율 행동	파일	LLM 모델
1	Planner의 의도 분류 + execution_plan	<code>agents/planner-agent.ts</code>	gpt-4o-mini
2	RAG 도메인 자동 라우팅	<code>agents/legal-rag-agent.ts:selectDomains</code>	(LLM 없이 모드 기반)
3	LLM 쿼리 리라이팅	<code>agents/legal-rag-agent.ts:rewriteQuery</code>	gpt-4o-mini
4	Self-RAG confidence 평가	<code>rag/search.ts:selfRagAssess</code>	gpt-4o-mini

#	자율 행동	파일	LLM 모델
5	Corrective RAG 자동 재검색	<code>rag/search.ts:correctiveRagSearch</code>	gpt-4o-mini
6	LLM Reranker (top 20 → top 5)	<code>rag/search.ts:llmRerank</code>	gpt-4o-mini
7	LLM Relevance Filter (Naver 결과)	<code>agents/local-context-light-agent.ts:llmRelevanceFilter</code>	gpt-4o-mini
8	Decision Agent verdict 합성	<code>agents/decision-agent.ts</code>	gpt-4o-mini
9	Fallback chain 자동 분기	<code>agents/competition-density-agent.ts</code>	(코드 분기)

4. 자율적이지 않은 부분 (한계)

#	한계	영향
1	Agent 호출 순서 하드코딩 (route.ts)	Planner의 execution_plan이 실제 실행에 영향 X
2	새 API/도구 발견 불가	LLM이 새 도구를 등록·사용 불가
3	Agent 추가/제거 시 코드 수정 필요	Hot-swap 불가
4	사용자 피드백 미반영	👍👎 없음, 학습 루프 없음
5	세션 간 메모리 없음	stateless, 매 분석 독립
6	단일 도구 호출	실패 시 자동 재시도/우회 없음 (Competition Density 4단 fallback만 예외)

가장 큰 모순

"Planner가 plan을 만들지만 누구도 그 plan을 따르지 않는다"

현재 Planner output:

```
{
  "execution_plan": [
```

```
{ "agent": "registry", "priority": "critical", "notes": "..." },
{ "agent": "market_data", "priority": "normal", "notes": "..." },
{ "agent": "building_register", "priority": "optional", "notes": "..." }
]
```

근데 route.ts:

```
// 무조건 이 순서 (priority 무시)
await runMarketDataAgent(...);
await runBuildingRegisterAgent(...);
await runRegistryAgent(...);
```

→ execution_plan은 표시용으로만 사용. 진정한 Level 3가 되려면 이 plan을 따라야 함.

5. 자율화 로드맵 — 4단계

● Phase A — 진짜 Planner 실행 (1주)

목표: execution_plan을 실제 실행 흐름에 반영.

구현:

- route.ts에 Planner-driven dispatcher
- critical → normal → optional 순서로 await
- timeout budget 추적 → optional skip
- UI에 priority 배지 + 실행 결과 시각화

효과:

- Level 2.5 → **Level 3 진입**
- 같은 모드라도 사용자 질문에 따라 다른 plan
- 발표 시 "동적 실행 순서 결정" 시연 가능

● Phase B — ReAct Agent Loop (2주)

목표: 각 Agent가 단일 호출이 아닌 반복 루프.

Agent: "Location Context 시작"

↓

[1] VWorld 호출 → 실패 → LLM "Naver 시도"

[2] Naver Geocoder → 부분 결과 → LLM "주소 정규화 후 재시도"

[3] 정규화 VWorld → 성공

프레임워크: OpenAI Assistants API (`tool_choice="auto"` + while loop) 또는 LangGraph.

효과: Level 4 (Self-correcting) 진입.

● Phase C — Multi-Agent Collaboration (2주)

목표: Agent 간 메시지 패싱.

Decision Agent: "이 카페는 정화구역 의심"

→ School Zone Agent에 추가 호출 요청

↓

School Zone Agent 재실행 (VWorld POI로)

→ 5건 잡힘

↓

Decision Agent 재판단: "정화구역 영향 악함, GO"

프레임워크: AutoGen, CrewAI, LangGraph multi-agent.

효과: Agent 협업 패턴.

● Phase D — Self-improving (1개월+)

목표: 사용자 피드백 학습.

사용자 🗨️ + "임대료 정보가 빠졌어요"

↓

시스템 학습:

- 이 모드+업종 조합에서 "임대료" 누락 패턴
- 다음 분석부터 `missing_aspects`에 자동 포함

↓

다음 분석 자동 개선

기술: RLHF, prompt 자동 진화, 사용자 행동 로깅.

6. 발표 시 사용 가능한 정확한 표현

✅ 사실에 부합

"터무니는 Level 2 도구 사용 + 부분적 Level 3 계획.
LLM 8개 결정 포인트에서 자율 판단.
Self-RAG + Corrective + Reranker는 Level 4 요소를 일부 적용."

❌ 과장 (피해야)

- "완전 자율 Agent"
- "AGI 수준"
- "Planner가 모든 흐름 통제"

✓ 솔직하면서 강한 메시지

"단순 규칙 기반이 아닙니다.
LLM이 8개 결정 지점에서 자율 판단하고,
Self-RAG로 자체 평가 + Corrective로 재시도까지 합니다.
다만 'Planner가 진짜 지휘자' (Level 3)는 다음 단계 로드맵에 있습니다."

7. Phase A 적용 후 비교

항목	현재	Phase A 적용 후
자율성 Level	2 + 부분 3	3
Planner의 plan	표시만	실제 실행 반영
같은 모드 다른 순서	불가	사용자 질문에 따라 동적
Agent skip 가능	불가	optional은 timeout 시 skip
UI 가시화	없음	priority 배지 + 실행 결과

8. 결론

- 터무니는 단순 규칙 기반이 아닌 **Level 2+** 시스템
- LLM 8개 결정 포인트가 자율성의 핵심 근거
- 가장 큰 모순: **Planner** 출력이 실제 실행에 영향 X
- Phase A 적용 시 Level 3 진입 + 발표 임팩트 큼

부록 — 자율성 측정 가능 메트릭 (향후)

- 실행 결정 분기 수: 8개 (LLM 결정 지점)
- LLM 호출 다양성: 같은 입력에 다른 plan 비율
- Self-RAG 발동 비율 + confidence 분포
- Corrective 재검색 발동 비율
- Fallback chain 깊이 분포

작성자: 터무니 팀 관련 문서:

- RAG + Multi-Agent 기술 상세
- 발표 시연 시나리오